

ForexAI

Updated Software Engineering Documentation

SRS + System Architecture + UML Sequences + Implementation Plan

Version 1.4

Date: 12 September 2026

Status: Development baseline | Implementation through Task 12 complete

Technology baseline: React + TypeScript | ASP.NET Core / C# / .NET 9 | Python + FastAPI +
LangGraph | PostgreSQL

Purpose

This document updates the original ForexAI software requirements and architecture into an implementation-ready baseline. It preserves the original objectives: React UX, C# application host, Python model and agent logic, PostgreSQL storage, near-live market data and trading signals, human acceptance/rejection of signals, session/short-term/long-term memory, low-cost operation, security and privacy, support for 3-10 users, testing, and online deployment.

Document control

Version	Date	Change	Status
1.0	08 Sep 2026	Initial SRS and architecture baseline	Superseded
1.1	09 Sep 2026	Added implementation architecture, domain-to-application mapping, task sequence, dependency rules and development gates	Current

1. System Vision

ForexAI is a multi-agent forex intelligence assistant. It collects and normalizes market information, performs technical/fundamental/quantitative analysis, evaluates risk, and produces a structured trading signal for human review. The MVP is a decision-support system; automatic broker execution is out of scope until a separate, tested execution capability is intentionally added.

2. Product Goals

- Provide near-live market information with a target delay below 5 seconds and a hard operational ceiling of 1 minute.
- Generate explainable, structured trading signals rather than free-form chatbot answers.
- Allow the user to accept or reject a signal and retain the decision as an auditable event.
- Maintain session, short-term and long-term memory without forcing a separate vector database in the MVP.
- Keep the initial operating cost as low as practical.
- Separate application hosting, AI/ML execution, persistence, and presentation so providers can be replaced later.
- Support an initial load of approximately 3-10 users.
- Run as an online, shareable application once the local MVP is stable.

3. Functional Requirements

ID	Requirement	Acceptance Measure	Priority
FR-MKT-01	The system shall ingest near-live forex market data.	Dashboard shows latest available quote/candle and timestamp; delay target <5s, ceiling <=60s.	Must
FR-MKT-02	The system shall normalize incoming market data into a consistent internal model.	Provider-specific payloads are converted into the domain market model before analysis.	Must
FR-ANL-01	The system shall execute multi-agent market analysis.	At minimum technical, fundamental, quant/ML, risk and decision roles can produce structured outputs.	Must
FR-ANL-02	The system shall preserve agent run inputs/outputs and status.	Each run has an identifier, timestamp, status and structured result stored for	Must

		audit/debugging.	
FR-SIG-01	The system shall create a structured trading signal.	Signal contains direction, confidence, entry, stop-loss, take-profit, risk/reward, timeframe and reasoning.	Must
FR-SIG-02	The system shall support human signal approval/rejection.	Only signals in UnderReview can transition to Accepted or Rejected.	Must
FR-MEM-01	The system shall maintain session, short-term and long-term memory.	Memory records can be stored and retrieved by user/session/context.	Must
FR-UX-01	The dashboard shall allow users to select a trading pair and review analysis.	User can choose a supported pair and see the latest signal and contributing agent results.	Must
FR-SEC-01	The system shall enforce agent data access boundaries.	Agents receive only the data/tools required for their role; secrets are not placed in prompts.	Must
FR-AUD-01	The system shall record important user/system decisions.	Signal acceptance/rejection and security-sensitive actions produce audit entries.	Should

4. Non-Functional Requirements

ID	Requirement	Acceptance Measure
NFR-PERF-01	Near-live freshness	Target <5s, maximum 60s from source timestamp to dashboard update.
NFR-SEC-01	Security	Secrets stored outside source code; authenticated endpoints; least-privilege service credentials.
NFR-SEC-02	Prompt security	Untrusted text is treated as data; system instructions and secrets are not exposed to agents.
NFR-REL-01	Resilience	Provider/API failures return controlled errors and do not

		corrupt domain state.
NFR-MNT-01	Maintainability	Clear separation of Domain, Application, Infrastructure and API projects.
NFR-TEST-01	Testability	Domain business rules covered by unit tests; later layers receive integration/API tests.
NFR-COST-01	Cost	MVP uses low-cost/local services where practical; paid inference can be swapped for another provider.
NFR-SCAL-01	Capacity	Initial deployment supports 3-10 concurrent users and does not require high-scale distributed infrastructure.
NFR-DEP-01	Deployment	Application can run as a shareable web application after local acceptance testing.

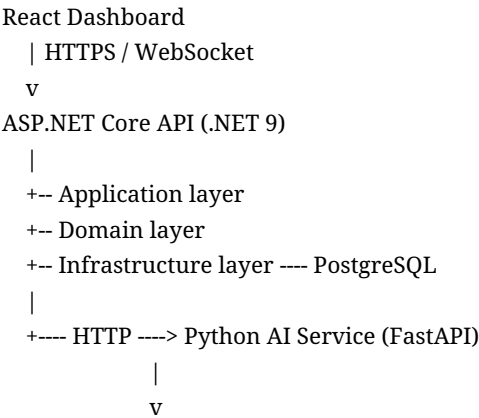
5. Scope and Boundaries

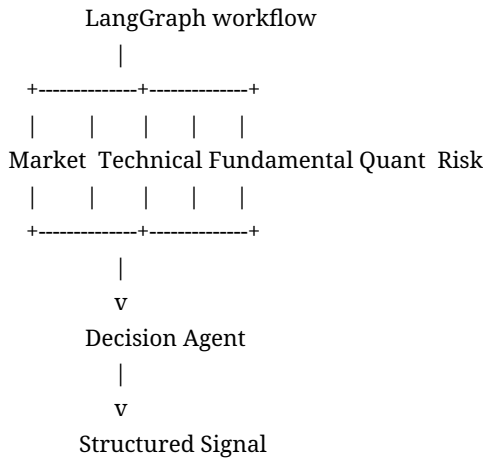
In scope for the MVP: market-data ingestion, pair selection, analysis workflow, structured signals, human review, memory, auditability, dashboard presentation, PostgreSQL persistence, and configurable LLM/model providers.

Out of scope for the MVP: unattended live broker execution, leveraged account management, production-grade high availability, large multi-tenant scale, and a guaranteed-profit trading strategy. Backtesting and paper trading are planned extensions.

6. System Architecture

The architecture uses four primary technology boundaries. React owns presentation. ASP.NET Core owns the application/API boundary and business workflow. Python owns agent orchestration and ML/model integration. PostgreSQL owns persistent application state and analysis history.





Architecture rule: Domain must remain independent of Infrastructure, API, Python, OpenAI, Hugging Face, LangGraph and PostgreSQL. Infrastructure implements persistence and external integrations behind application/domain interfaces.

7. UML Sequence Definitions

7.1 Analyze a Pair

User -> React -> C# API -> Application -> AI Service -> LangGraph agents -> Risk -> Decision -> C# -> PostgreSQL -> React.

7.2 Approve/Reject a Signal

User -> React -> C# API -> Application -> TradingSignal state transition -> PostgreSQL audit -> React confirmation.

7.3 Live Market Update

Market provider -> C# market-data adapter -> normalization -> PostgreSQL -> API/WebSocket -> React dashboard.

7.4 Memory Lifecycle

User/session -> Application -> memory service -> PostgreSQL; relevant memory is read before analysis and updated after an agent run.

8. Domain Model Baseline

- **ForexPair:** identifies the tradable pair and whether it is active.
- **MarketCandle:** validated OHLCV data with timeframe and timestamp.
- **MarketAnalysis:** normalized market interpretation such as trend, volatility and regime.
- **TradingSignal:** recommendation with confidence, prices, risk/reward, reasoning and a controlled lifecycle.
- **Enums:** SignalDirection, SignalStatus and Timeframe provide controlled values rather than arbitrary strings.

9. Development Status

Task	Area	Status	Definition of Done
Task 1	Solution + project structure + test framework	Complete	Build succeeds; project dependency cycle removed; 6 tests pass.
Task 2	Domain model + business rules	In progress	Domain entities/enums complete; validation and lifecycle tests pass.
Task 3	Application layer	Next	Use cases, DTOs and ports/interfaces compile and have application tests.
Task 4	Infrastructure + PostgreSQL	Planned	EF Core/Npgsql configured, migrations created, repositories tested.
Task 5	Python AI service + LangGraph	Planned	FastAPI service returns versioned structured agent workflow result.
Task 6	React dashboard	Planned	Dashboard consumes API and supports signal review.
Task 7	Integration, backtesting, deployment	Planned	End-to-end test, model evaluation and shareable deployment complete.

10. Task 3 - Application Layer

Objective: build the application/use-case layer before adding PostgreSQL or Python. This layer translates user intentions into application actions and defines interfaces for things the outer layers must provide.

Why: it prevents controllers from becoming business logic, prevents PostgreSQL code from leaking into use cases, and gives the future Python AI service a clean boundary.

10.1 Create Application folders

```
ForexAI.Application
|
+-- Contracts
|   +-- Analysis
|   +-- Signals
|   +-- MarketData
|
+-- Interfaces
|   +-- IMarketDataProvider.cs
|   +-- ITradingSignalRepository.cs
|   +-- IMarketAnalysisRepository.cs
|   +-- IAIAnalysisClient.cs
|   +-- IClock.cs
|
+-- Services
|   +-- AnalyzeMarketService.cs
|   +-- TradingSignalService.cs
|
+-- Exceptions
|
```

Create only these folders now. Do not add EF Core or HTTP implementation classes to Application.

10.2 Create the application contracts

These DTOs are boundary models. They exist so the API does not expose internal domain entities directly.

```
namespace ForexAI.Application.Contracts.Analysis;

public sealed record AnalyzeMarketRequest(
    Guid ForexPairId,
    string Symbol,
    Timeframe Timeframe);

public sealed record AnalyzeMarketResult(
    Guid AnalysisId,
    Guid SignalId,
    string Symbol,
    string Direction,
    decimal Confidence,
    decimal EntryPrice,
```

```
decimal StopLoss,
decimal TakeProfit,
decimal RiskReward,
string Reasoning);
```

Why: contracts define what the Application layer needs from callers and what it returns. We can change the internal entity model without breaking the API contract immediately.

10.3 Create the AI boundary interface

```
using ForexAI.Application.Contracts.Analysis;

namespace ForexAI.Application.Interfaces;

public interface IAIAnalysisClient
{
    Task<AnalyzeMarketResult> AnalyzeAsync(
        AnalyzeMarketRequest request,
        CancellationToken cancellationToken = default);
}
```

Why: Application must not reference Python, FastAPI, LangGraph or OpenAI. It only knows that an AI analysis capability exists.

10.4 Create repository interfaces

```
using ForexAI.Domain.Entities;

namespace ForexAI.Application.Interfaces;

public interface ITradingSignalRepository
{
    Task AddAsync(TradingSignal signal, CancellationToken cancellationToken = default);
    Task<TradingSignal?> GetByIdAsync(Guid id, CancellationToken cancellationToken = default);
    Task SaveAsync(TradingSignal signal, CancellationToken cancellationToken = default);
}

public interface IMarketDataProvider
{
    Task<IReadOnlyCollection<MarketCandle>>
        GetRecentCandlesAsync(
            Guid forexPairId,
            int count,
            CancellationToken cancellationToken = default);
}
```

Why: the Application layer defines what it requires. Infrastructure later decides whether data comes from PostgreSQL, a broker API, a file, or a test fake.

10.5 Create the analysis service

```
using ForexAI.Application.Contracts.Analysis;
using ForexAI.Application.Interfaces;
using ForexAI.Domain.Entities;
```

```
namespace ForexAI.Application.Services;

public sealed class AnalyzeMarketService
{
    private readonly IAIAnalysisClient _aiClient;
    private readonly ITradingSignalRepository _signalRepository;

    public AnalyzeMarketService(
        IAIAnalysisClient aiClient,
        ITradingSignalRepository signalRepository)
    {
        _aiClient = aiClient;
        _signalRepository = signalRepository;
    }

    public async Task<AnalyzeMarketResult> ExecuteAsync(
        AnalyzeMarketRequest request,
        CancellationToken cancellationToken = default)
    {
        if (request.ForexPairId == Guid.Empty)
            throw new ArgumentException(
                "Forex pair ID is required.",
                nameof(request));

        if (string.IsNullOrEmpty(request.Symbol))
            throw new ArgumentException(
                "Symbol is required.",
                nameof(request));

        var aiResult = await _aiClient.AnalyzeAsync(
            request,
            cancellationToken);

        var signal = new TradingSignal(
            request.ForexPairId,
            aiResult.Direction,
            aiResult.Confidence,
            aiResult.EntryPrice,
            aiResult.StopLoss,
            aiResult.TakeProfit,
            aiResult.RiskReward,
            request.Timeframe,
            aiResult.Reasoning);

        signal.SubmitForReview();

        await _signalRepository.AddAsync(
            signal,
            cancellationToken);

        await _signalRepository.SaveAsync(
```

```

        signal,
        cancellationToken);

    return new AnalyzeMarketResult(
        signal.Id,
        signal.ForexPairId,
        request.Symbol,
        signal.Direction,
        signal.Confidence,
        signal.EntryPrice,
        signal.StopLoss,
        signal.TakeProfit,
        signal.RiskReward,
        signal.Timeframe,
        signal.Status.ToString(),
        signal.Reasoning);
    }
}

```

Important: this is deliberately a thin first implementation. Once the Python contract is fixed, we will map the returned analysis into a TradingSignal domain entity, validate it, persist it, and return the DTO.

10.6 Application-layer dependency rule

```

ForexAI.Application
-> ForexAI.Domain

```

ForexAI.Application MUST NOT reference:

- ForexAI.Api
- ForexAI.Infrastructure
- Python
- FastAPI
- LangGraph
- OpenAI SDK
- Npgsql / Entity Framework

10.7 Task 3 tests

- Test AnalyzeMarketService calls IAiAnalysisClient once for a valid request.
- Test the returned result is passed back to the caller.
- Test a cancellation token is forwarded to the AI boundary.
- Later: test invalid AI output is rejected when domain mapping is implemented.

10.8 Task 3 Definition of Done

- Application compiles targeting net9.0.
- Application references Domain only.
- DTOs exist for the analyze-market use case.
- IAiAnalysisClient and repository interfaces compile.
- AnalyzeMarketService can be tested without PostgreSQL, Python or OpenAI.
- Application tests pass.

- No Infrastructure or API dependencies have been added to Application.

11. Task 3 Implementation Checklist in Visual Studio

Step	Action	Reason
1	Right-click ForexAI.Application -> Add -> New Folder -> Contracts, Interfaces, Services, Exceptions.	Create a clean namespace structure before adding code.
2	Add AnalyzeMarketRequest.cs and AnalyzeMarketResult.cs.	Define the use-case boundary independent of HTTP and persistence.
3	Add IAIAnalysisClient.cs.	Reserve the boundary that will later call the Python service.
4	Add ITradingSignalRepository.cs and IMarketDataProvider.cs.	Define persistence/data needs without selecting technology.
5	Add AnalyzeMarketService.cs.	Create the first application use case and keep controller logic thin.
6	Add application tests with fakes/mocks.	Verify orchestration without external dependencies.
7	Build Solution, then run dotnet test.	Confirm the new layer has not broken the existing domain tests.
8	Commit to Git.	Create a stable checkpoint before Infrastructure is introduced.

12. Next Tasks After Task 3

Task 4 will implement PostgreSQL persistence in Infrastructure. We will create the EF Core DbContext, entity configurations, migrations, and repository implementations against the previously approved ERD. The database will not be designed ad hoc from the tables; it will be mapped from the domain and persistence requirements.

Task 5 will create the Python FastAPI service and LangGraph workflow. The Python service will expose a structured contract so C# can treat the model/agent subsystem as a replaceable provider. OpenAI, Hugging Face, local models, and the user's own models will be behind provider abstractions.

Task 6 will create the React dashboard and connect it to the ASP.NET API. Signal approval/rejection will travel through C# and the domain state machine rather than being implemented solely in the frontend.

13. Architecture Decisions

ADR-001: C# is the application host

Uses the user's stronger C# background and provides a clear API/business boundary.

ADR-002: Python owns AI/ML

Provides access to the Python ML ecosystem and LangGraph while keeping AI concerns isolated.

ADR-003: PostgreSQL is the system of record

Stores market data, analyses, signals, agent runs, memory and audit information in one relational baseline.

ADR-004: LLM provider abstraction

Prevents OpenAI, Hugging Face or local-model choice from leaking into every agent.

ADR-005: Human-in-the-loop before execution

Reduces operational risk and matches the original acceptance/rejection requirement.

ADR-006: No vector DB in MVP

Session/short-term/long-term memory can begin with PostgreSQL; introduce vector retrieval only when retrieval quality or scale demonstrates a need.

14. Requirement Traceability

Requirement	Architecture	Implementation Task	Verification
FR-MKT-01	Market data adapter + API + React update	Task 4 / Task 6	Freshness test
FR-ANL-01	Python AI service + LangGraph	Task 5	Agent workflow integration test
FR-SIG-01	TradingSignal + application service	Task 2 / Task 3	Domain + application tests
FR-SIG-02	TradingSignal state machine + API	Task 2 / Task 6	Accept/reject API tests
FR-MEM-01	Memory service + PostgreSQL	Task 4 / Task 5	Memory read/write tests
FR-SEC-01	API auth + least privilege + agent tool boundaries	Task 3+	Security tests and review
NFR-COST-01	Provider abstraction + local development	Task 3 / Task 5	Deployment cost review
NFR-SCAL-01	Stateless API + simple deployment baseline	Task 4+	3-10 user load test

15. Current Developer Checkpoint

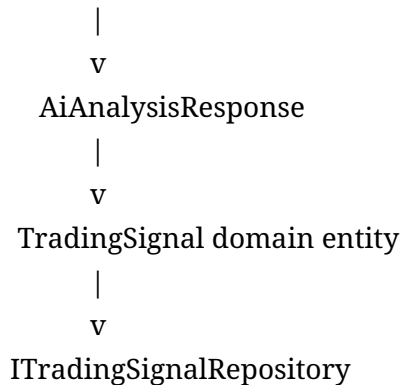
Current status before this revision: Task 1 through the C# vertical slice and the initial Python service have been implemented. Task 6 and Task 7 are now complete: Python runs as an independent VS Code project, FastAPI exposes /health and /analysis, LangGraph is installed, the first graph contains a Technical Agent node, and the async graph test passes after pytest package-path and asyncio configuration were added. Task 8 is the next development milestone.

16. Task 3 Engineering Amendment - Authoritative Implementation

This section is the authoritative Task 3 implementation specification. It refines the earlier Task 3 examples so that the AI service returns an AI analysis result, while the Application layer remains responsible for creating and validating the TradingSignal domain entity. This keeps IDs, state transitions and business invariants inside the application/domain boundary.

16.1 Task 3 target architecture

C# API -> AnalyzeMarketService -> IAiAnalysisClient -> Python AI service



Reason: an external AI provider must not generate or control persistence identifiers or domain state. The C# application receives model output, validates it, creates the domain signal, moves it to UnderReview, and persists it.

16.2 Folder structure to create

Inside ForexAI.Application create:

```

Contracts/
  Analysis/
    AnalyzeMarketRequest.cs
    AiAnalysisResponse.cs
    AnalyzeMarketResult.cs
  Interfaces/
    IAiAnalysisClient.cs
    ITradingSignalRepository.cs
  Services/
    AnalyzeMarketService.cs
  
```

16.3 AnalyzeMarketRequest.cs

```

using ForexAI.Domain.Enums;

namespace ForexAI.Application.Contracts.Analysis;

public sealed record AnalyzeMarketRequest(
    Guid ForexPairId,
    string Symbol,
    Timeframe Timeframe);
  
```

Why: the request is the input to the application use case. Timeframe is typed with the domain enum so callers cannot send arbitrary timeframe values.

16.4 AiAnalysisResponse.cs

```
using ForexAI.Domain.Enums;

namespace ForexAI.Application.Contracts.Analysis;

public sealed record AiAnalysisResponse(
    SignalDirection Direction,
    decimal Confidence,
    decimal EntryPrice,
    decimal StopLoss,
    decimal TakeProfit,
    decimal RiskReward,
    string Reasoning);
```

Why: this represents the provider-facing analysis result. It does not contain a TradingSignal ID or status because those belong to the application/domain lifecycle.

16.5 AnalyzeMarketResult.cs

```
using ForexAI.Domain.Enums;

namespace ForexAI.Application.Contracts.Analysis;

public sealed record AnalyzeMarketResult(
    Guid SignalId,
    Guid ForexPairId,
    string Symbol,
    SignalDirection Direction,
    decimal Confidence,
    decimal EntryPrice,
    decimal StopLoss,
    decimal TakeProfit,
    decimal RiskReward,
    Timeframe Timeframe,
    string Status,
    string Reasoning);
```

Why: this is the application output. The caller can receive a safe, read-oriented view of the created signal without receiving the full domain object.

16.6 IAiAnalysisClient.cs

```
using ForexAI.Application.Contracts.Analysis;

namespace ForexAI.Application.Interfaces;

public interface IAiAnalysisClient
{
    Task<AiAnalysisResponse> AnalyzeAsync(
```

```
AnalyzeMarketRequest request,
CancellationToken cancellationToken = default);
}
```

Why: this is the clean boundary between the C# Application layer and the Python/LLM/ML subsystem. Application depends on the capability, not on FastAPI, LangGraph, OpenAI or a specific model provider.

16.7 ITradingSignalRepository.cs

```
using ForexAI.Domain.Entities;

namespace ForexAI.Application.Interfaces;

public interface ITradingSignalRepository
{
    Task AddAsync(
        TradingSignal signal,
        CancellationToken cancellationToken = default);

    Task<TradingSignal?> GetByIdAsync(
        Guid id,
        CancellationToken cancellationToken = default);

    Task SaveAsync(
        TradingSignal signal,
        CancellationToken cancellationToken = default);
}
```

Why: Application defines the persistence capability it requires. Infrastructure will later implement this interface using EF Core and PostgreSQL.

16.8 AnalyzeMarketService.cs

```
using ForexAI.Application.Contracts.Analysis;
using ForexAI.Application.Interfaces;
using ForexAI.Domain.Entities;
using ForexAI.Domain.Enums;

namespace ForexAI.Application.Services;

public sealed class AnalyzeMarketService
{
    private readonly IAiAnalysisClient _aiClient;
    private readonly ITradingSignalRepository _signalRepository;

    public AnalyzeMarketService(
        IAiAnalysisClient aiClient,
        ITradingSignalRepository signalRepository)
    {
        _aiClient = aiClient;
        _signalRepository = signalRepository;
    }
}
```

```

public async Task<AnalyzeMarketResult> ExecuteAsync(
    AnalyzeMarketRequest request,
    CancellationToken cancellationToken = default)
{
    if (request.ForexPairId == Guid.Empty)
        throw new ArgumentException(
            "Forex pair ID is required.",
            nameof(request));

    if (string.IsNullOrEmpty(request.Symbol))
        throw new ArgumentException(
            "Symbol is required.",
            nameof(request));

    var aiResult = await _aiClient.AnalyzeAsync(
        request,
        cancellationToken);

    if (!Enum.TryParse<SignalDirection>(
        aiResult.Direction,
        true,
        out var direction))
    {
        throw new InvalidOperationException(
            $"AI returned an unsupported direction: {aiResult.Direction}");
    }

    if (!Enum.TryParse<Timeframe>(
        aiResult.Timeframe,
        true,
        out var timeframe))
    {
        throw new InvalidOperationException(
            $"AI returned an unsupported timeframe: {aiResult.Timeframe}");
    }

    var signal = new TradingSignal(
        request.ForexPairId,
        direction,
        aiResult.Confidence,
        aiResult.EntryPrice,
        aiResult.StopLoss,
        aiResult.TakeProfit,
        aiResult.RiskReward,
        timeframe,
        aiResult.Reasoning);

    signal.SubmitForReview();

    await _signalRepository.AddAsync(
        signal,

```

```

        cancellationToken);

    return new AnalyzeMarketResult(
        signal.Id,
        signal.ForexPairId,
        request.Symbol,
        signal.Direction.ToString(),
        signal.Confidence,
        signal.EntryPrice,
        signal.StopLoss,
        signal.TakeProfit,
        signal.RiskReward,
        signal.Timeframe.ToString(),
        signal.Status.ToString(),
        signal.Reasoning);
    }
}

```

Why: this service is the orchestration point. It validates the request, invokes AI, validates provider output, creates the domain TradingSignal, applies the domain lifecycle rule, persists it through an interface, and returns a DTO.

16.9 Application tests - use a hand-written fake first

Avoid adding a mocking framework yet. A small fake keeps the learning objective visible and avoids unnecessary dependencies.

```

private sealed class FakeAiAnalysisClient : IAiAnalysisClient
{
    public bool WasCalled { get; private set; }
    public AiAnalysisResponse Response { get; set; } = new(
        "Buy", 0.80m, 1.1650m, 1.1600m, 1.1750m, 2.0m,
        "FourHours", "Bullish technical confirmation.");

    public Task<AiAnalysisResponse> AnalyzeAsync(
        AnalyzeMarketRequest request,
        CancellationToken cancellationToken = default)
    {
        WasCalled = true;
        return Task.FromResult(Response);
    }
}

private sealed class FakeTradingSignalRepository : ITradingSignalRepository
{
    public TradingSignal? SavedSignal { get; private set; }

    public Task AddAsync(
        TradingSignal signal,
        CancellationToken cancellationToken = default)
    {
        SavedSignal = signal;
        return Task.CompletedTask;
    }
}

```

```

    }

    public Task<TradingSignal?> GetByIdAsync(
        Guid id,
        CancellationToken cancellationToken = default)
        => Task.FromResult(SavedSignal);

    public Task SaveAsync(
        TradingSignal signal,
        CancellationToken cancellationToken = default)
    {
        SavedSignal = signal;
        return Task.CompletedTask;
    }
}

```

Test cases: the service calls the AI client, creates a signal, moves it to UnderReview, saves it, returns the generated signal ID, rejects unsupported AI direction/timeframe, and propagates cancellation.

16.10 Task 3 Definition of Done

- ForexAI.Application references ForexAI.Domain only.
- No EF Core, Npgsql, HTTP client, OpenAI SDK, LangGraph or Python package is installed in Application.
- The three contract records and two interfaces compile.
- AnalyzeMarketService converts provider output into a TradingSignal domain entity.
- The domain controls the signal state transition to UnderReview.
- The repository is called only through ITradingSignalRepository.
- Application tests pass without PostgreSQL or Python running.
- Solution build remains green and all existing domain tests continue to pass.
- A Git commit is created before starting Infrastructure/Task 4.

16.11 Exact Visual Studio work order

Step	Action	Reason
A	Create the folders under ForexAI.Application.	Namespace organization and separation of contracts, ports and services.
B	Add the three contract records.	Define stable input/output boundaries before implementing orchestration.
C	Add IAiAnalysisClient and ITradingSignalRepository.	Apply dependency inversion: Application owns required capabilities; outer layers implement them.
D	Add AnalyzeMarketService.	Implement the first application use case without persistence technology.
E	Add the fakes and tests.	Prove orchestration without running PostgreSQL, Python,

		LangGraph or an LLM.
F	Build and test.	Confirm architectural boundaries and regression safety.
G	Commit: feat(application): add market analysis use case.	Create a stable milestone before Infrastructure is introduced.

17. Current Implementation Status and Engineering Amendments

This section is authoritative for the implementation state as of 12 September 2026. It reconciles earlier planning examples with the code structure actually established during development.

Task	Scope	Status	Evidence / next action
Task 1	Solution structure and project dependencies	Complete	All C# projects target .NET 9; dependency cycle fixed; solution builds.
Task 2	Domain model and business rules	Complete	ForexPair, MarketCandle, MarketAnalysis, TradingSignal, enums, lifecycle rules, and unit tests are in place.
Task 3	Application layer	Complete	Contracts, IAIAnalysisClient, ITradingSignalRepository, AnalyzeMarketService, and application tests are in place.
Task 4	Infrastructure + PostgreSQL	Complete	EF Core/Npgsql, DbContext, entity configurations, repository, migration, and database update are working.
Task 5	ASP.NET Core vertical slice	Complete	Health endpoint and POST /api/Analysis tested; a signal can be generated and persisted.
Task 6	Python + FastAPI service	Complete	Python service moved to VS Code; virtual environment, FastAPI, schemas, /health and /analysis work.
Task 7	LangGraph foundation	Complete	Graph state, Technical Agent node, graph service, FastAPI integration, and async pytest test are working.
Task 8	Full multi-agent graph + real technical logic	Next	Add explicit state, real technical calculations, and the remaining

			agent nodes before selecting an LLM provider.
--	--	--	---

17.1 Authoritative cross-service contract

The C# application request now carries the strongly typed timeframe. The Python service receives a JSON representation of the request, while the C# application remains responsible for domain typing and lifecycle state.

C# request contract:

```
public sealed record AnalyzeMarketRequest(
    Guid ForexPairId,
    string Symbol,
    Timeframe Timeframe
);
```

Python provider-facing response contract:

```
class AiAnalysisResponse(BaseModel):
    direction: str
    confidence: float
    entry_price: float
    stop_loss: float
    take_profit: float
    risk_reward: float
    reasoning: str
```

The provider response does not determine the TradingSignal ID, lifecycle status, or requested timeframe. The C# Application layer creates the domain TradingSignal with request.Timeframe and moves it to UnderReview before persistence.

17.2 Repository contract amendment

ITradingSignalRepository remains a three-method contract because that is the established implementation in the current codebase:

```
Task AddAsync(TradingSignal signal, CancellationToken cancellationToken = default);
Task<TradingSignal?> GetByIdAsync(Guid id, CancellationToken cancellationToken = default);
Task SaveAsync(TradingSignal signal, CancellationToken cancellationToken = default);
```

Infrastructure implements all three methods. AnalyzeMarketService explicitly calls AddAsync followed by SaveAsync. This is now the authoritative behaviour for the current milestone; any later repository refactor must be recorded as an architecture decision rather than silently changing the contract.

17.3 Python development environment amendment

Python is intentionally maintained as a separate VS Code project rather than as Python items embedded in the Visual Studio C# solution. The C# and Python applications communicate over HTTP.

Local development topology:

Visual Studio 2022

ForexAI.Api <https://localhost:7296>

ForexAI.Application

ForexAI.Domain

ForexAI.Infrastructure

ForexAI.Tests

VS Code

forexai-ai <http://127.0.0.1:8000>

app/agents

app/api

app/graphs

app/schemas

app/services

tests/

This separation supports independent tooling, testing, deployment, and future hosting of the AI/model subsystem.

17.4 Python test configuration amendment

The Python project requires an explicit pytest project-root path and asyncio support. The current pytest.ini baseline is:

```
[pytest]
pythonpath = .
asyncio_mode = auto
```

Async graph tests are marked with `@pytest.mark.asyncio`. This resolves the initial `ModuleNotFoundError` for app and the subsequent async test execution error without modifying application import paths.

17.5 Verified LangGraph baseline

The current graph is intentionally small. It proves orchestration before real model calls are introduced:

```
START
|
v
Technical Agent
|
v
END
```

The next graph revision will preserve this testable design while adding parallel or conditionally routed analysis nodes.

18. Task 8 — Full Multi-Agent Graph and Real Technical Analysis

Objective: evolve the one-node LangGraph foundation into the planned multi-agent architecture without prematurely coupling the agents to a specific LLM provider.

18.1 Subtask A — Expand the graph state

Add explicit fields for market context and each analysis result: `technical_analysis`, `fundamental_analysis`, `quant_prediction`, `risk_assessment`, `final_decision`, plus request context. Reason: the state should make the data contract between agents visible and testable.

18.2 Subtask B — Introduce a Market Data node

Create a deterministic Market Data node that accepts supplied OHLCV input and validates the basic shape. Reason: later Technical and Quant agents should consume structured market data rather than inventing values.

18.3 Subtask C — Upgrade the Technical Agent

Implement deterministic indicators first: price range/return, EMA/SMA, RSI, MACD and ATR as the first practical set. Return structured output containing trend, momentum, volatility and an explanatory summary. Reason: technical calculations must be reproducible and testable; the LLM should interpret results rather than calculate them unreliably.

18.4 Subtask D — Add Fundamental, Quant and Risk nodes as controlled stubs

Add typed node outputs with deterministic placeholder logic before connecting external providers. Reason: the graph topology should be proven independently of news APIs, proprietary models and LLM credentials.

18.5 Subtask E — Add the Decision Agent

Combine the structured agent outputs and produce a final decision object. Explicitly preserve supporting evidence and conflicting evidence. Reason: a multi-agent assistant should be able to show disagreement rather than hide it.

18.6 Task 8 Definition of Done

- `ForexAnalysisState` contains the planned analysis/result fields.
- Market Data, Technical, Fundamental, Quant and Risk nodes execute successfully in tests.
- Decision node produces a deterministic structured final decision from the supplied state.
- No agent depends directly on OpenAI, Hugging Face, PostgreSQL or the C# API.
- Graph tests pass under pytest with asyncio enabled.
- The graph can be invoked from FastAPI through the existing `/analysis` endpoint.

19. Next Architecture Milestones

19.1 Completed implementation milestones

- **Task 1-6: Foundation and services:** Complete. Visual Studio C# solution, domain model, application layer, PostgreSQL persistence, C# API vertical slice, separate VS Code Python/FastAPI service.
- **Task 7: LangGraph foundation:** Complete. Graph state, Technical Agent node, graph service, FastAPI integration, and async pytest configuration are working.
- **Task 8: Real technical-analysis foundation:** Complete. Market data schema, EMA/RSI/ATR calculations, deterministic Technical Agent behaviour, and tests are in place.
- **Task 9: Full multi-agent graph:** Complete. Technical, Fundamental, Quant, Risk, and Decision nodes are orchestrated through a shared LangGraph state, including disagreement/risk-rejection scenarios.
- **Task 10: Market-data abstraction:** Complete. MarketDataProvider, MarketDataService, mock provider, and graph integration are implemented. Real external market ingestion remains a later integration task.
- **Task 11: ML model abstraction:** Complete. QuantModel interface, scikit-learn adapter, model artifact workflow, Quant Agent integration, and tests are implemented.
- **Task 12: LLM provider abstraction:** Complete. LLMProvider, OpenAIProvider, generic HTTP provider, provider factory, environment configuration, and contract tests are implemented. The next step is connecting the provider to agent reasoning.

19.2 Next implementation milestones

- **Task 13: LLM-enabled Technical Agent:** Feed deterministic technical evidence to the configurable LLM provider and validate structured reasoning before writing to graph state.
- **Task 14: C# ↔ Python AI integration:** Replace FakeAiAnalysisClient with a production-shaped PythonAiAnalysisClient using HTTP, request/response validation, timeout handling, and error mapping.
- **Task 15: React dashboard foundation:** Create the React/TypeScript dashboard, API client, market view, analysis view, and signal review UI.
- **Task 16: Real market-data ingestion:** Implement a real provider adapter, normalization, freshness timestamps, retry/rate-limit behaviour, and the sub-5-second target/one-minute ceiling instrumentation.
- **Task 17: Memory and audit history:** Implement session, short-term, and long-term memory plus auditable agent-run and user-decision history.
- **Task 18: Testing, backtesting, and deployment:** Add broader integration/e2e tests, backtesting, Docker/CI, security hardening, and low-cost online deployment for the 3-10 user target.

20. Version History

1.4 | 12 September 2026

Records completion of Task 12 LLM provider abstraction and establishes the authoritative next milestones for LLM-agent integration, C#↔Python integration, React, real market ingestion, memory, and deployment.

Version	Date	Change	Status
1.0	08 Sep 2026	Initial expanded SRS, class diagram and ERD baseline.	Superseded
1.1	09 Sep 2026	Application-layer implementation plan and architecture details.	Superseded
1.2	09 Sep 2026	C# infrastructure, PostgreSQL, API and development guidance.	Superseded
1.3	12 Sep 2026	Reconciled authoritative contracts; recorded completed C# vertical slice, Python/FastAPI, LangGraph foundation, VS Code Python workflow, pytest fixes, and Task 8 plan.	Current

21. Authoritative Engineering Note

The v1.4 baseline reflects the implementation decisions established during development. Where earlier planning sections use different task numbering or provisional examples, the completed implementation milestones in Section 19.1 and the current codebase are authoritative. In particular, the C# application uses a strongly typed Timeframe in AnalyzeMarketRequest; ITradingSignalRepository retains AddAsync, GetByIdAsync and SaveAsync; and Python is maintained as a separate VS Code service communicating with the C# application over HTTP.